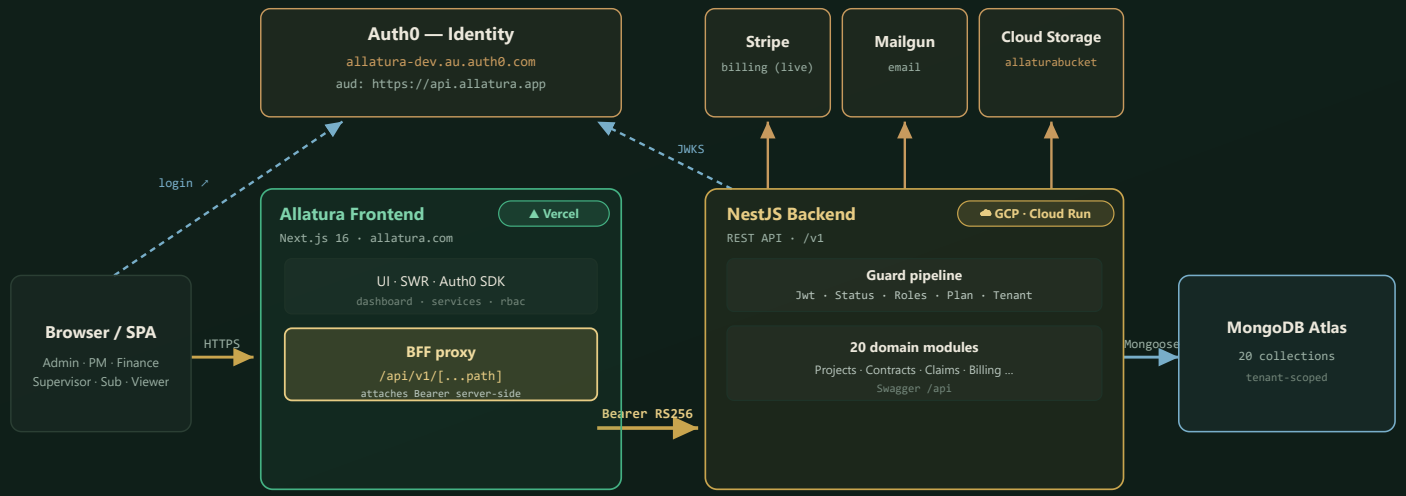
01

CURRENT SYSTEM ARCHITECTURE · WHAT WE BUILT & DEPLOYED

Components, platforms & connections



RESPONSIBILITIES & PLATFORMS

- **Frontend** — Next.js on **Vercel** · `allatura.com`
- **Backend** — NestJS on **GCP Cloud Run** · `plutus-api-…run.app/v1`
- **Identity** — Auth0 Universal Login · RS256 + JWKS
- **Data** — MongoDB Atlas · 20 tenant-scoped collections
- **Files · Billing · Email** — Cloud Storage · Stripe (live) · Mailgun

request path · identity / data · integrations · dashed = OAuth / verify

`allatura.com/dashboard`

App UI — dashboard
drop screenshot

`console.cloud.google.com · Cloud Run · plutus-api`

GCP Cloud Run
drop screenshot

`cloud.mongodb.com · Atlas`

MongoDB Atlas
drop screenshot

02

IDENTITY · TENANT ISOLATION · REQUEST FLOW

One request, end to end — where security is enforced

PRIMARY FLOW — USER → RESPONSE

1. Login on **allatura.com** → Auth0 **Universal Login** (`/authorize`)
2. Callback → tokens sealed in an **encrypted httpOnly cookie** — never in JS
3. App calls **`/api/v1/*`** → BFF attaches **`Bearer <RS256>`**
4. AuthN — backend verifies RS256 against Auth0 **JWKS**
5. AuthZ — role, status & tenant read **from the database**, not the token
6. Tenant-scoped query hits **Atlas** / returns a **GCS signed URL** → response

Authentication proves **who you are** (Auth0 signature). **Authorisation** decides **what you may do** — and that is enforced in **our database**, never trusted from the token.

TENANT ISOLATION & EXTERNAL FLOWS

Tenant isolation

Every document carries a `tenantId`; a scope interceptor blocks any cross-org id in the request → **403**. `jwtVersion` enables instant revocation.

Onboarding & invites MAILGUN

`First user → creates org, becomes` **`owner`** `· admin invites by email → /invite/<token> → joins tenant`

Billing webhooks STRIPE

`Stripe → /billing/webhook (signature-verified).` **`Stripe is source of truth`** `& retries →` **`eventually consistent`**

manage.auth0.com · Application + Login Action

Auth0 dashboard
drop screenshot

dashboard.stripe.com · Webhooks · Events

Stripe — webhook events
drop screenshot

03 DESIGN DECISIONS · NFRS · TRADE-OFFS

Why it's built this way — what each choice gained & cost

GUIDING PRINCIPLE

Maximise *security* and *delivery speed* for a small team — with a known path to *harden at scale*.

SECURITY SCALABILITY CONSISTENCY MAINTAINABILITY RELIABILITY

1 Auth0 + RS256 — token sealed server-side via the BFF

SECURITY

Gain XSS-safe tokens; SSO/MFA built in; rotatable keys.

Cost An extra network hop; reliance on Auth0.

2 Identity & tenant resolved in the DB, not the token

SECURITY CONSISTENCY

Gain No privilege escalation; one source of truth; instant revoke.

Cost A cached DB check on every request.

3 Multi-tenant modular monolith on Cloud Run

SCALABILITY MAINTAINABILITY

Gain One cheap deployment; fast iteration; scale-to-zero.

Cost Shared blast radius; cold-start latency.

4 Offload billing & files; precompute financials

RELIABILITY PERFORMANCE

Gain Stripe/GCS manage retries & bytes; dashboards read instantly.

Cost Eventual consistency; external dependencies.